Null Object

eason: 4 · Enterprise · Category: Enterprise · Level: Intermediate · Build with Waqa

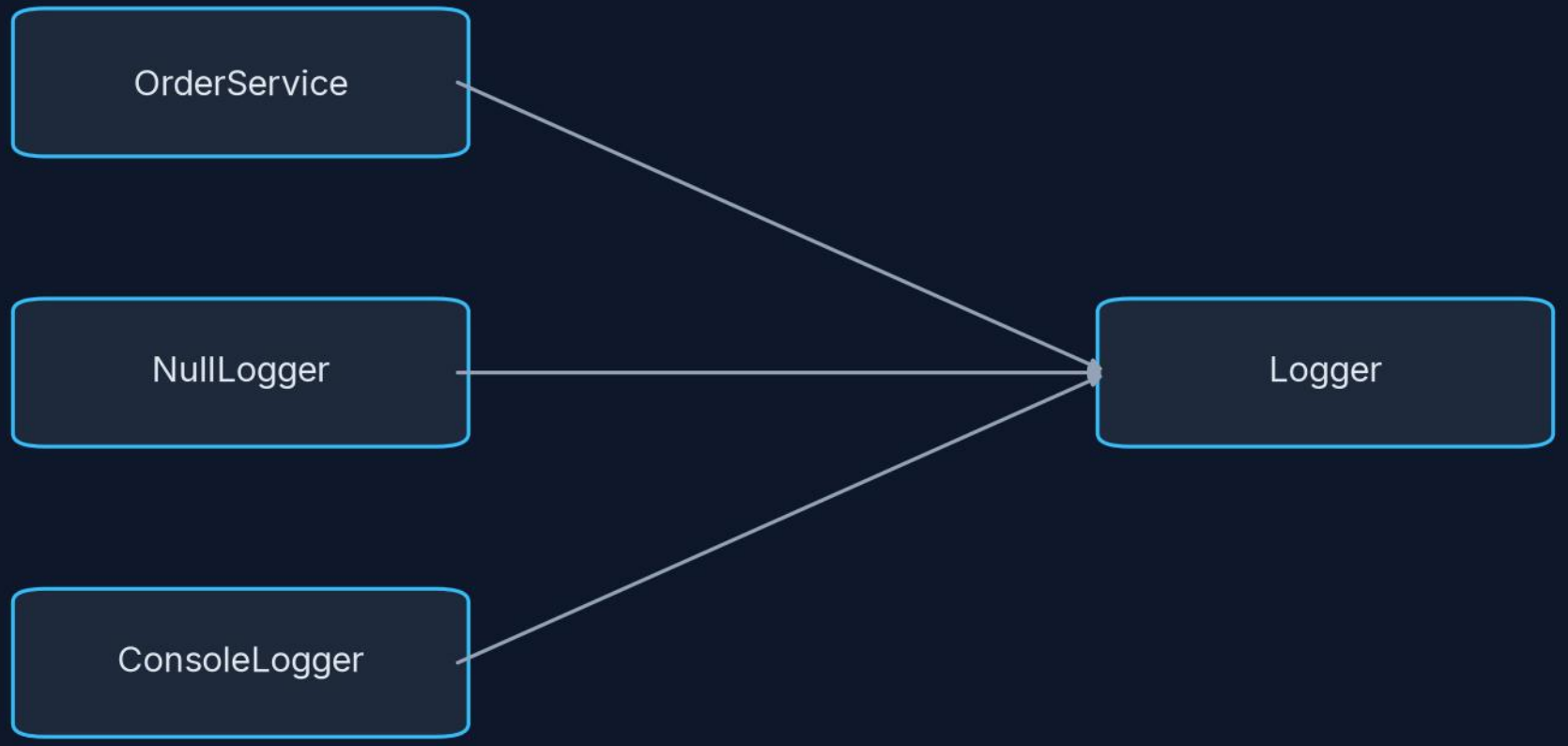
Today's Hook

- Today: Null Object as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Null Object structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Null Object
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/enterprise/nullobject/OrderService.java
package com.buildwithwaqar.patterns.enterprise.nullobject;

interface Logger {
    void info(String msg);
}

final class ConsoleLogger implements Logger {
    private final StringBuilder sink;
    ConsoleLogger(StringBuilder sink) { this.sink = sink; }
    public void info(String msg) { sink.append(msg).append('\n'); }
}

final class NullLogger implements Logger {
    static final NullLogger INSTANCE = new NullLogger();
    public void info(String msg) { /* no-op */ }
}

/** Prefer Null Object over null checks at call sites. */
public final class OrderService {
    private final Logger logger;
```

Implementation (2)

```
    public OrderService(Logger logger) {
        this.logger = logger == null ? NullLogger.INSTANCE : logger;
    }

    public String place(String id) {
        logger.info("placing " + id);
        return "ok:" + id;
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.concurrency.activeobject;
public final class Demo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Active Object walkthrough ===");
    try (var ao = new ActiveObject()) {
      System.out.println("add2=" + ao.add(2).get());
      System.out.println("add3=" + ao.add(3).get());
      System.out.println("get=" + ao.get().get());
    }
  }
}
```

Walkthrough Output

```
=== Null Object walkthrough ===
ok:ord-1
ok:ord-2
log=placing ord-2
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Null Object
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Null Object → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.